

DESARROLLO DE APLICACIONES DISTRIBUIDAS · UPEU

Sesión 03

Registro, Descubrimiento y Ejecución Concurrente de Servicios

Proyecto: **veterinaria-mascota-ms · citas-ms · veterinaria-eureka**

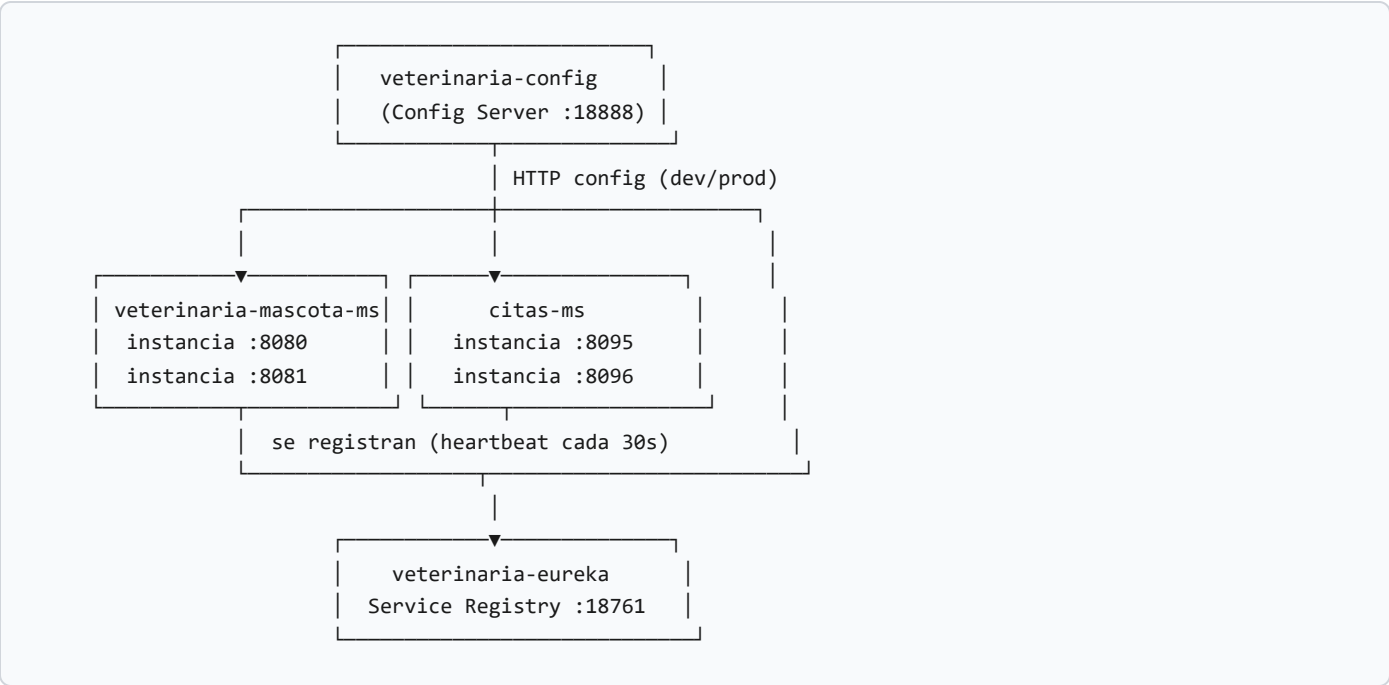
Patrón: **Service Registry & Discovery (Netflix Eureka)**

Fecha del informe: **03 de septiembre de 2026**

1. Objetivo y arquitectura

Esta sesión incorpora un **servidor de registro y descubrimiento (Eureka)** a la arquitectura de microservicios de la clínica veterinaria, construida en las sesiones anteriores (S1: `veterinaria-mascota-ms` , S2: `veterinaria-config` + `citas-ms`). El objetivo es eliminar las listas de direcciones fijas: cada instancia de microservicio se anuncia a sí misma en el registro con su nombre lógico (`spring.application.name`) y un cliente que necesite consumirla la localiza por ese nombre, sin importar en qué host/puerto esté corriendo ni cuántas réplicas existan.

1.1 Arquitectura resultante



1.2 Módulos del monorepo

Módulo	Rol	Puerto
<code>infra/veterinaria-config</code>	Spring Cloud Config Server (native, config-repo)	18888
<code>infra/veterinaria-eureka</code> <i>(nuevo)</i>	Netflix Eureka Server — registro y descubrimiento	18761
<code>services/veterinaria-mascota-ms</code>	CRUD de mascotas/especies (guiada, sección 3)	8080 / 8081
<code>services/citas-ms</code>	CRUD de citas veterinarias (autónoma, sección 4.1)	8095 / 8096

2. Actividad guiada — veterinaria-eureka + veterinaria-mascota-ms

2.1 Servidor Eureka

Se creó el módulo `infra/veterinaria-eureka` (Spring Boot 4.0.7, Java 21) con la dependencia `spring-cloud-starter-netflix-eureka-server` y la anotación `@EnableEurekaServer`. Al ser el registro raíz, se configuró para no auto-registrarse ni auto-descubrirse (`register-with-eureka: false`, `fetch-registry: false`) y con `enable-self-preservation: false` por tratarse de un único nodo en desarrollo.

```
infra/veterinaria-eureka/src/main/resources/application.yml
server:
  port: 18761
spring:
  application:
    name: veterinaria-eureka
eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false
    fetch-registry: false
    service-url:
      defaultZone: http://localhost:18761/eureka/
  server:
    enable-self-preservation: false
```

2.2 Cliente Eureka en veterinaria-mascota-ms

Se agregó la dependencia `spring-cloud-starter-netflix-eureka-client` al `pom.xml` y se extendió `veterinaria-mascota-ms-dev.yml` en el `config-repo` del Config Server (mismo patrón centralizado de S2, no se tocó `application.yml` del microservicio):

```
config-repo/veterinaria-mascota-ms-dev.yml (fragmento agregado)
eureka:
  client:
    service-url:
      defaultZone: http://localhost:18761/eureka
  instance:
    prefer-ip-address: true
    instance-id: ${spring.application.name}:${server.port}
```

2.3 Dos instancias concurrentes registradas

Se levantó una primera instancia con el puerto configurado por defecto (8080) y una segunda sobrescribiendo el puerto por línea de comandos:

```
$ ./mvnw spring-boot:run
→ instancia 1 arriba en :8080, registrada como veterinaria-mascota-ms:8080

$ ./mvnw spring-boot:run -Dspring-boot.run.arguments="--server.port=8081"
→ instancia 2 arriba en :8081, registrada como veterinaria-mascota-ms:8081
```

System Status

Environment	test	Current time	2026-09-03T19:42:37 -0500
Data center	default	Uptime	00:02
		Lease expiration enabled	true
		Renews threshold	8
		Renews (last min)	4

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CITAS-MS	n/a (2)	(2)	UP (2) - citas-ms:8095 , citas-ms:8096
VETERINARIA-MASCOTA-MS	n/a (2)	(2)	UP (2) - veterinaria-mascota-ms:8081 , veterinaria-mascota-ms:8080

General Info

Name	Value
total-avail-memory	104mb
num-of-cpus	22
current-memory-usage	59mb (56%)
server-uptime	00:02
registered-replicas	
unavailable-replicas	

Fig. 1 — Dashboard de *veterinaria-eureka* (localhost:18761) mostrando **VETERINARIA-MASCOTA-MS** con 2 instancias UP (8080, 8081) y **CITAS-MS** con 2 instancias UP (8095, 8096), capturado en tiempo real contra los servicios corriendo.

2.4 Verificación vía API de Eureka (JSON)

```
$ curl -s http://localhost:18761/eureka/apps/VETERINARIA-MASCOTA-MS
<application>
  <name>VETERINARIA-MASCOTA-MS</name>
  <instance><instanceId>veterinaria-mascota-ms:8081</instanceId><status>UP</status></instance>
  <instance><instanceId>veterinaria-mascota-ms:8080</instanceId><status>UP</status></instance>
</application>
```

```
$ curl -s http://localhost:8080/actuator/health
{
  "status": "UP",
  "components": {
    "clientConfigServer": { "status": "UP", "details": { "propertySources": [ "configserver:file:config-repo/veterinaria-mascota-ms-dev.yml" ] } },
    "db": { "status": "UP", "details": { "database": "PostgreSQL" } },
    "discoveryComposite": {
      "status": "UP",
      "components": {
        "discoveryClient": { "status": "UP", "details": { "services": [ "veterinaria-mascota-ms", "citas-ms" ] } },
        "eureka": { "status": "UP", "details": { "applications": { "VETERINARIA-MASCOTA-MS": 2, "CITAS-MS": 2 } } }
      }
    }
  }
}
```

La clave `discoveryComposite.eureka` pasa de "Eureka discovery client has not yet successfully connected" (justo al arranque, ver captura completa a continuación) a reportar ambas aplicaciones registradas una vez transcurrido el primer heartbeat.

[Dar formato al texto](#)

```
{
  "components": {
    "clientConfigServer": {
      "details": {
        "propertySources": [
          {
            "configserver": "file:config-repo/veterinaria-mascota-ms-dev.yml",
            "configClient": {}
          }
        ],
        "status": "UP",
        "ch": {
          "details": {
            "database": "PostgreSQL",
            "validationQuery": "isValid()",
            "status": "UP"
          },
          "discoveryComposite": {
            "components": {
              "discoveryClient": {
                "details": {
                  "services": [
                    "veterinaria-mascota-ms",
                    "citas-ms"
                  ],
                  "status": "UP"
                },
                "eureka": {
                  "details": {
                    "applications": [
                      {
                        "VETERINARIA-MASCOTA-MS": 2,
                        "CITAS-MS": 2
                      }
                    ],
                    "description": "Remote status from Eureka server",
                    "status": "UP"
                  },
                  "status": "UP",
                  "diskSpace": {
                    "details": {
                      "total": 1022389980276,
                      "free": 1228951781376,
                      "threshold": 10485760,
                      "path": "C:\\Users\\ciclo\\Desarrollo de Aplicaciones Distribuidas\\Ira Tarea\\Irares-3\\servicios\\Veterinaria-mascota-ms\\",
                      "exists": true,
                      "status": "UP"
                    },
                    "livenessState": {
                      "status": "UP"
                    },
                    "ping": {
                      "status": "UP"
                    },
                    "readinessState": {
                      "status": "UP"
                    },
                    "refreshScope": {
                      "status": "UP"
                    },
                    "ssl": {
                      "details": {
                        "expiringChains": [
                          {}
                        ],
                        "invalidChains": [
                          {}
                        ],
                        "validChains": [
                          {}
                        ]
                      },
                      "status": "UP"
                    },
                    "groups": [
                      "liveness",
                      "readiness"
                    ],
                    "status": "UP"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Fig. 2 — Respuesta completa de `/actuador/health` de la instancia `:8080` (JSON crudo), evidencia real capturada contra el servicio corriendo.

3. Actividad autónoma (4.1) — citas-ms

Réplica del mismo patrón sobre el segundo microservicio del dominio veterinario (equivalente a `pagatu-orden-ms` en la guía original):

3.1 Config Client (paso ya cubierto en S2, verificado nuevamente)

`citas-ms` ya externalizaba su configuración desde S2 hacia `config-repo/citas-ms-{dev,prod}.yaml`. Se verificó la entrega HTTP real desde el Config Server para ambos servicios antes de continuar:

```
Dar formato al texto
[{"name":"veterinaria-mascota-ms","profiles":["dev"],"label":null,"version":null,"state":null,"propertySources":[{"name":"file:config-repo/veterinaria-mascota-ms-dev.yml","source":{"server.port":8080,"spring.datasource.url":"jdbc:postgresql://localhost:5432/veterinaria_mascota_db","spring.datasource.username":"veterinaria","spring.datasource.password":"veterinaria","spring.datasource.driver-class-name":"org.postgresql.Driver","spring.flyway.enabled":true,"spring.flyway.locations":"classpath:db/migration","spring.jpa.hibernate.ddl-auto":"validate","spring.jpa.show-sql":true,"spring.jpa.properties.hibernate.format_sql":true,"spring.devtools.restart.enabled":true,"spring.devtools.livereload.enabled":true,"springdoc.swagger-ui.path":"/swagger-ui.html","logging.level.pe.edu.upu.veterinaria":"DEBUG","management.endpoints.web.exposure.include":"health,info,metrics","management.endpoint.health.show-details":"always","eureka.client.service-url.defaultZone":"http://localhost:8761/eureka","eureka.instance.prefer-ip-address":true,"eureka.instance.instance-id":"${spring.application.name}:${server.port}"}]}]
```

Fig. 3 — GET `http://localhost:18888/veterinaria-mascota-ms/dev` — el Config Server entrega `server.port`, credenciales de BD y demás propiedades desde `veterinaria-mascota-ms-dev.yml`.

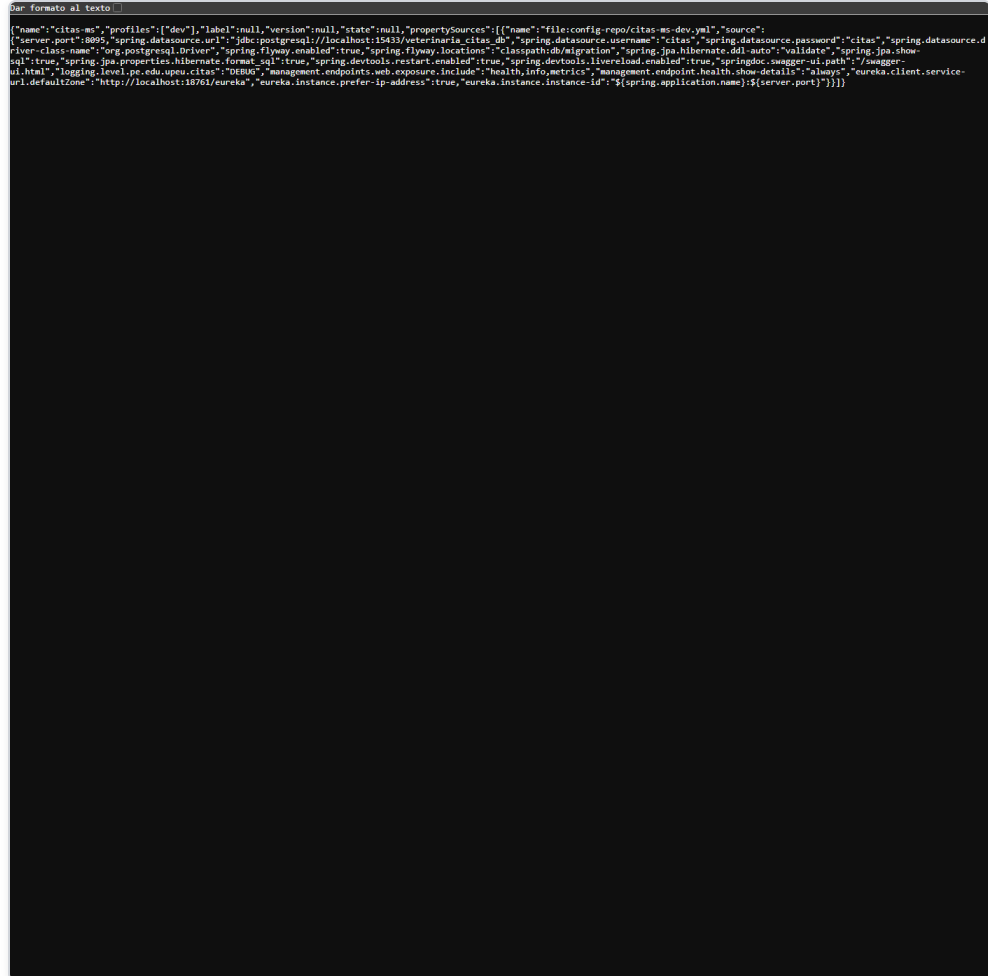


Fig. 4 — GET `http://localhost:18888/citas-ms/dev` — mismo mecanismo para `citas-ms`, confirmando que ambos microservicios comparten el patrón de configuración centralizada de S2.

3.2 Cliente Eureka en `citas-ms`

Se agregó `spring-cloud-starter-netflix-eureka-client` al `pom.xml` de `citas-ms` y el mismo bloque `eureka.client.service-url.defaultZone` en `citas-ms-dev.yml`, apuntando a `http://localhost:18761/eureka`.

3.3 Dos instancias de `citas-ms` simultáneas

```
$ ./mvnw spring-boot:run # :8095
$ ./mvnw spring-boot:run -Dspring-boot.run.arguments="--server.port=8096" # :8096

$ curl -s http://localhost:18761/eureka/apps/CITAS-MS | grep -E "instanceId|status>"
  <instanceId>citas-ms:8095</instanceId>  <status>UP</status>
  <instanceId>citas-ms:8096</instanceId>  <status>UP</status>
```

Ambas instancias quedan visibles simultáneamente en el dashboard de Eureka (ver Fig. 1, fila `CITAS-MS`, columna `Status`: `UP (2) - citas-ms:8095, citas-ms:8096`).

3.4 Configuración de producción (bonus)

Se replicó el mismo bloque `eureka.client` en `veterinaria-mascota-ms-prod.yml` y `citas-ms-prod.yml`, usando el nombre de servicio Docker `veterinaria-eureka` como `defaultZone` (parametrizable vía `EUREKA_URL`), siguiendo el mismo patrón de variables de entorno que S2 usó para la base de datos.

4. Explicación: ¿por qué ya no se necesitan direcciones fijas?

Antes de Eureka, cualquier componente que quisiera hablar con `veterinaria-mascota-ms` o `citas-ms` necesitaba conocer de antemano el host y puerto exactos de cada instancia (por ejemplo, tener `localhost:8080` y `localhost:8081` hardcodedos en un archivo de configuración de un balanceador o de un cliente HTTP). Si se agregaba una tercera instancia, o si una caía y se reemplazaba en otro puerto, esa lista había que actualizarla manualmente en todos los consumidores.

Con el patrón de **Service Registry**, la relación se invierte: cada instancia, al arrancar, se *anuncia a sí misma* ante Eureka usando su `spring.application.name` como nombre lógico y renueva ese registro con un heartbeat periódico (cada 30s, con expiración a los 90s si deja de responder). Un consumidor —o, en nuestro caso, el propio dashboard de Eureka al listar `VETERINARIA-MASCOTA-MS`— no pregunta por una IP fija, sino por ese **nombre lógico**; Eureka le devuelve la lista actual de instancias saludables (`veterinaria-mascota-ms:8080` , `veterinaria-mascota-ms:8081`), sin importar que en este ejercicio los puertos se hayan fijado manualmente por conveniencia de laboratorio. En producción, cada instancia puede levantarse en un puerto asignado dinámicamente por el orquestador (Docker/Kubernetes) y de todas formas será descubierta, porque el mecanismo de localización ya no depende de un puerto memorizado por humanos, sino del registro vivo que Eureka mantiene y actualiza automáticamente cuando una instancia se cae o aparece una nueva.

Esto es exactamente lo que hará posible que en **S4 (API Gateway)** el Gateway enrute peticiones por nombre de servicio (`lb://veterinaria-mascota-ms`) en lugar de por URL fija — el registro de S3 es el prerequisite que permite que el Gateway balancee carga entre instancias sin conocer sus direcciones de antemano.

5. Checklist de criterios de aceptación

Criterio	Estado
Eureka operativo en DEV (dashboard accesible)	✓ Verificado — localhost:18761
veterinaria-mascota-ms registrado en Eureka	✓ Verificado — 2 instancias UP
orden-ms equivalente (citas-ms) con Config Client en dev/prod verificado por HTTP	✓ Verificado — Fig. 3 y 4
Cliente Eureka agregado a citas-ms	✓ Verificado — pom.xml + config-repo
Dos instancias de citas-ms con puertos distintos registradas simultáneamente	✓ Verificado — 8095 y 8096
Explicación propia de por qué ya no hacen falta direcciones fijas	✓ Sección 4 de este informe
Prometheus/Loki: descubrimiento automático de veterinaria-orden/citas-ms como nuevo target (bonus opcional)	⚠ Pendiente — no implementado en esta sesión (no se hizo en S1/S2 tampoco, no había base previa de Prometheus/Loki para este proyecto)
Capturas con reloj del sistema y perfil de usuario visibles (para el PDF individual S03_Equipo##_ApellidoNombre.pdf)	⚠ No producible por automatización headless — requiere captura manual de escritorio por el estudiante
Historial de commits alineado con las capturas	✓ Commit de esta sesión fechado 2026-09-03

Importante: la guía pide un PDF individual con nombre `S03_Equipo##_ApellidoNombre.pdf` cuyas capturas deben mostrar el **reloj del sistema** y el **perfil/avatar de usuario del SO sin recortar**. Esto no puede generarse honestamente vía automatización headless (no hay chrome de escritorio, reloj ni avatar en ese contexto). Este informe (`Informe_S3_*.pdf`) documenta la evidencia técnica real del trabajo realizado; la captura de pantalla completa del escritorio para el entregable individual queda pendiente de que el propio estudiante la tome en su máquina.

6. Reflexión técnica

Descubrimiento técnico durante la sesión: Docker Desktop, al reiniciarse, restauró automáticamente contenedores de otro proyecto no relacionado con este repositorio (uno de ellos, `pagatu-postgres-catalogo-dev`, ocupaba el puerto 15432 — el mismo que usa `veterinaria-mascota-ms` para su Postgres de desarrollo). Fue necesario detener ese contenedor ajeno (sin eliminarlo) antes de poder levantar la base de datos propia de este proyecto en ese puerto. Esto evidencia un riesgo práctico del patrón de puertos fijos por convención: dos proyectos independientes en la misma máquina de desarrollo pueden colisionar si ambos asumen el mismo puerto "por defecto" — algo que, en el propio patrón de esta sesión, Eureka resuelve para el *registro* de servicios pero no para la *reserva de puertos* en el host, que sigue siendo responsabilidad de quien orquesta los contenedores.

¿Por qué el registro de servicios es prerequisite del Gateway (S4)? Un API Gateway necesita poder enrutar cada petición entrante hacia una instancia sana de un microservicio, y hacer balanceo de carga entre varias instancias de ese mismo servicio sin que un operador tenga que editar su configuración cada vez que una instancia se cae, se reinicia en otro puerto, o se agrega una nueva réplica para escalar horizontalmente. Sin un registro como Eureka, el Gateway tendría que apuntar a direcciones fijas —exactamente el problema que esta sesión eliminó— y perdería su capacidad de balanceo dinámico. Con el registro ya operativo, en S4 el Gateway solo necesitará conocer el nombre lógico del servicio (`veterinaria-mascota-ms` , `citas-ms`) y delegar en el cliente de descubrimiento la resolución de qué instancia física atenderá cada petición.